

AI 驱动的工程知识同步平台

AI-Driven Engineering Knowledge Sync Platform

1. 😞 你在工作中遇到了什么痛点？过去你是怎么处理的？你希望通过 AI 达到什么效果？What work pain points are you facing? How did you tackle them before? What's your vision for an AI-powered solution?

在大型系统开发过程中，数据库表结构（Table）、字段（Column）、索引、配置以及业务特征会频繁变化，但相关文档通常无法及时更新。

常见问题

- 新成员 onboarding 时难以理解当前系统结构
- 跨团队沟通成本高，经常需要重复确认字段用途
- 数据库影响范围难以快速分析
- Schema 变更遗漏影响 API、报表或下游服务
- ERD（Entity Relationship Diagram）容易过期
- 配置中心（Apollo / Consul）新增配置无人记录（有些需求通过各自工单维护，或者 Devops 车单仅此），导致环境行为难追踪

这些问题引发这类问题

- 历史变更不好追踪
- 文档与实际代码不一致
- 不同服务之间的依赖关系不透明

目前方式

目前团队主要依赖以下方式维护文档：

- 开发者手动更新 Lark Document / Lark Wiki / Excel / Draw.io
- Review PR 时顺便检查数据库变更

- 文档更新延迟，开发与实际数据库结构不同步

但这些方式高度依赖人为自觉，容易遗漏。

AI 解决方案的方向

我的目标是打造一个 AI + 自动化驱动文档同步方案，当代码仓库检测到数据库相关变更时，系统能够自动：

- 分析数据库变更内容
- 判断新增 / 修改 / 删除的字段与表
- 自动生成变更摘要
- 更新 Lark Document
- 自动生成 ERD
- 记录历史版本
- 分析潜在影响服务
- 在需要时进一步分析 Apollo / Consul 配置

最终希望达到：“代码即文档（Code as Documentation）” 的效果，减少人工维护成本，并提升团队协作效率。

2. 你是如何解决的？How did you solve it?

我的方案核心是“代码驱动分析 + AI 文档生成”。

整体流程如下：

Step 1：监听代码仓库变更

系统监听：

- GitLab PR （release 分支）
- 业务模块代码
- 车单

重点分析：

- SQL Migration
- JPA Entity
- MyBatis Mapper

- Flyway / Liquibase
- ORM Model
- DDL Script
- 配置文件 (Apollo / Consul)

Step 2: AI 分析数据库变更

AI 会根据代码差异 (Diff) 判断:

- 哪些表被修改
- 哪些字段新增 / 删除
- 字段类型是否变化
- 是否影响索引
- 是否存在 breaking change
- 是否影响下游服务
- 是否影响 API

例如: `ALTER TABLE user ADD COLUMN phone VARCHAR(20);`

AI 可自动生成:

- 新增字段: phone
- 类型: VARCHAR(20)
- 风险等级: Low
- 可能影响: 用户注册、Profile API

Step 3: 自动更新 Lark 文档

系统自动:

- 更新数据字典
- 更新表结构说明
- 更新字段定义
- 更新 ERD 图
- 生成 ChangeLog
- 记录版本历史

Step 4: 生成影响分析

系统进一步分析:

- 哪些服务引用该表
- 哪些 API 使用该字段
- 哪些配置存在关联
- 哪些 Kafka / MQ Topic 可能受影响

Step 5: AI 总结与通知

AI 自动生成：

- 业务摘要
- 风险提示
- Reviewer 建议
- 测试建议

并同步到：

- Lark
- Slack
- Email
- CI/CD Pipeline

3. 预估的落地收益？ Estimated benefits from implementation?

预估收益如下：

- 文档维护时间 ▼ 70%~90%
- Onboarding 时间 ▼ 50%
- 新人 Onboarding ↑ 50%
- 数据库变更和文档遗漏率 **大幅降低**
- ERD长期**保持最新状态**
- Release 风险 提早发现潜在影响
- 提升跨团队协作效率
- 减少线上事故风险
- 提高系统透明度与可维护性

长期来看：

- 可形成长期企业知识库
- AI 可逐渐学习团队命名规范与架构习惯
- 支持 AI 问答
- 支持智能影响分析
- 支持自动生成开发文档
- 支持架构治理与审计

4. 🐛 在使用新办法的过程中，有遇到新问题吗？如果有你是如何解决的？ Did any new issues arise while using the new approach? If so, how did you resolve them?

有，主要包括以下几个问题：

问题 1：AI 误判复杂 SQL

例如：

- 动态 SQL
- 多阶段 Migration
- Rename Column
- Legacy Script

解决方式：

- 引入规则引擎
- 增加 AST（Abstract Syntax Tree）解析
- 增加人工 Review Gate
- 高风险变更需人工确认

问题 2：AI 对业务语义理解不准确

例如：

Code block

```
1  status = 2
```

AI 无法直接知道业务含义。

解决方案：

- 结合代码上下文
- 分析 Enum / Constant Definition
- 建立业务术语词典（Business Glossary）

问题 3：不同团队代码风格不一致

解决方式：

- 建立统一 Migration 规范
- 定义 Metadata Annotation
- 标准化命名规则
- 建立 Plugin-based Analyzer 让系统具备扩展能力：
 - EF Plugin
 - SQL Parser Plugin
 - Flyway Plugin
 - Liquibase Plugin

问题 4：配置来源复杂

部分配置来自：

- Apollo
- Consul
- Environment Variables
- Kubernetes ConfigMap

解决方案：

建立统一 Config Discovery Layer。

问题 5：AI 成本控制

解决方式：

- 小模型处理普通 Diff

- 大模型只处理复杂分析
- Cache 重复结果
- 使用事件驱动降低无效分析

5. 🤔 在你看来，其他人在什么场景下也能使用这个方案？ In your opinion, in what scenarios can others also apply this method?

这个方案不仅适用于数据库文档同步，还可以扩展到：

- API 文档自动更新
- 微服务依赖分析
- Kafka / MQ Topic 关系图
- 配置中心变更分析
- Terraform / IaC 基础设施文档
- Kubernetes YAML 分析
- 安全审计
- 架构治理
- 自动生成 Release Notes
- 自动生成测试影响分析

适用于：

- 金融系统
- 电商平台
- SaaS 平台
- 微服务架构
- 大型企业系统
- 多团队协作开发环境

6. 💰 你的方案是否包含成本统计与控制规划？ Does your proposal include cost statistics and control plans?

方案包含：

- 只分析变更文件 (Incremental Analysis)
- 使用轻量模型处理普通 Diff
- 高风险分析才调用大型模型
- Cache 历史分析结果
- 定时批量处理减少 Token 成本

7. 你的方案有说明用户数据如何被安全处理（不滥用、不长期保留）吗？ Does your proposal specify how user data is securely processed (no abuse, no long-term retention)?

安全策略：

- 仅分析 Metadata 与 Schema
- 不读取敏感业务数据
- Token / Secret 自动 Mask
- 分析日志短期保留
- 支持企业内网模型
- 权限分级访问
- 不长期保留原始数据
- 审计日志
- 企业私有部署支持

8. 其他需要补充的 Additional Notes

这个方案最大的价值不只是“自动更新文档”，而是建立一个“工程知识自动沉淀系统”。

AI 不只是生成文档，而是逐步理解：

- 数据结构
- 服务依赖
- 业务关系
- 配置关联
- 系统影响范围

未来可以进一步扩展成：

- 企业 AI Knowledge Graph
- AI 架构分析助手
- AI Code Reviewer
- AI 运维影响分析平台

Closing Point

“真正的问题不是文档没人写，而是文档更新速度永远追不上代码变化速度。”

这个方案希望让 AI 成为开发流程的一部分，让系统能够自动理解变更、沉淀知识，并持续保持文档与系统真实状态同步。

最终目标是：让工程知识不再依赖人，而是自动持续演进。

“The real problem is not that engineers refuse to write documentation — it is that documentation can never keep up with the speed of code changes.”

This solution aims to make AI part of the engineering workflow, enabling systems to automatically understand changes, preserve knowledge, and continuously synchronize documentation with real system states.

The ultimate goal is: To make engineering knowledge evolve automatically instead of depending entirely on humans.